# VLIW Example 3

**Example** Suppose we have a VLIW that could issue two memory references, two FP operations, and one integer operation or branch in every clock cycle. Show an unrolled version of the loop `x[i] = x[i] + s` (see page 158 for the MIPS code) for such a processor. Unroll as many times as necessary to eliminate any stalls. Ignore delayed branches.

***Answer*** Figure 3.16 shows the code. The loop has been unrolled to make seven copies of the body, which eliminates all stalls (i.e., completely empty issue cycles), and runs in 9 cycles. This code yields a running rate of seven results in 9 cycles, or 1.29 cycles per result, nearly twice as fast as the two-issue superscalar of Section 3.2 that used unrolled and scheduled code.

The straightforward MIPS code, not scheduled for the pipeline, looks like this:

```
Loop:   L.D      F0,0(R1)     ;F0=array element
        ADD.D    F4,F0,F2     ;add scalar in F2
        S.D      F4,0(R1)     ;store result
        DADDUI   R1,R1,#-8    ;decrement pointer
                              ;8 bytes (per DW)
        BNE      R1,R2,Loop   ;branch R1!=R2
```

Let's start by seeing how well this loop will run when it is scheduled on a simple pipeline for MIPS with the latencies from Figure 3.2.

```
for (i=999; i>=0; i=i-1)
       x[i] = x[i] + s;
```

The straightforward MIPS code, not scheduled for the pipeline, looks like this:

```
Loop:   L.D     F0,0(R1)     ;F0=array element
        ADD.D   F4,F0,F2     ;add scalar in F2
        S.D     F4,0(R1)     ;store result
        DADDUI  R1,R1,#-8    ;decrement pointer
                             ;8 bytes (per DW)
        BNE     R1,R2,Loop   ;branch R1!=R2
```

Let's start by seeing how well this loop will run when it is scheduled on a simple pipeline for MIPS with the latencies from Figure 3.2.

| Instruction producing result | Instruction using result | Latency in clock cycles |
|---|---|---|
| FP ALU op | Another FP ALU op | 3 |
| FP ALU op | Store double | 2 |
| Load double | FP ALU op | 1 |
| Load double | Store double | 0 |

**Figure 3.2 Latencies of FP operations used in this chapter.** The last column is the number of intervening clock cycles needed to avoid a stall. These numbers are similar to the average latencies we would see on an FP unit. The latency of a floating-point load to a store is 0, since the result of the load can be bypassed without stalling the store. We will continue to assume an integer load latency of 1 and an integer ALU operation latency of 0.

# 4 Unrolled Loop Example

```
Loop:   L.D      F0,0(R1)
        ADD.D    F4,F0,F2
        S.D      F4,0(R1)      ;drop DADDUI & BNE
        L.D      F6,-8(R1)
        ADD.D    F8,F6,F2
        S.D      F8,-8(R1)     ;drop DADDUI & BNE
        L.D      F10,-16(R1)
        ADD.D    F12,F10,F2
        S.D      F12,-16(R1)   ;drop DADDUI & BNE
        L.D      F14,-24(R1)
        ADD.D    F16,F14,F2
        S.D      F16,-24(R1)
        DADDUI   R1,R1,#-32
        BNE      R1,R2,Loop
```

# Loop Unrolling in VLIW

```
Loop:   L.D     F0,0(R1)
        ADD.D   F4,F0,F2
        S.D     F4,0(R1)
        DADDUI  R1,R1,#-8

        BNE     R1,R2,Loop
```

| Instruction producing result | Instruction using result | Latency in clock cycles |
|---|---|---|
| FP ALU op | Another FP ALU op | 3 |
| FP ALU op | Store double | 2 |
| Load double | FP ALU op | 1 |
| Load double | Store double | 0 |

**Unrolled 7 times to avoid delays**

**7 results in 9 clocks, or 1.3 clocks per iteration (1.8X)**

**Average: 2.5 ops per clock, 50% efficiency**

| Memory reference 1 | Memory reference 2 | FP operation 1 | FP operation 2 | Integer operation/branch |
|---|---|---|---|---|
| L.D F0,0(R1) | L.D F6,-8(R1) | | | |
| L.D F10,-16(R1) | L.D F14,-24(R1) | | | |
| L.D F18,-32(R1) | L.D F22,-40(R1) | ADD.D F4,F0,F2 | ADD.D F8,F6,F2 | |
| L.D F26,-48(R1) | | ADD.D F12,F10,F2 | ADD.D F16,F14,F2 | |
| | | ADD.D F20,F18,F2 | ADD.D F24,F22,F2 | |
| S.D F4,0(R1) | S.D F8,-8(R1) | ADD.D F28,F26,F2 | | |
| S.D F12,-16(R1) | S.D F16,-24(R1) | | | DADDUI R1,R1,#-56 |
| S.D F20,24(R1) | S.D F24,16(R1) | | | |
| S.D F28,8(R1) | | | | BNE R1,R2,Loop |

**Figure 3.16 VLIW instructions that occupy the inner loop and replace the unrolled sequence.** This code takes 9 cycles assuming no branch delay; normally the branch delay would also need to be scheduled. The issue rate is 23 operations in 9 clock cycles, or 2.5 operations per cycle. The efficiency, the percentage of available slots that contained an operation, is about 60%. To achieve this issue rate requires a larger number of registers than MIPS would normally use in this loop. The VLIW code sequence above requires at least eight FP registers, while the same code sequence for the base MIPS processor can use as few as two FP registers or as many as five when unrolled and scheduled.